IPA 1/69

Variables & Reusable Modules

WEEK 12

var

local

output

module

Week 2 Agenda

WEEK 2

3 hours: ~60 min concept + ~120 min hands-on lab

- Modules overview — structure, submodules, root, registry
- Three variable kinds: input, output, locals — when to use each
- Input variables — defining, sensitive, type constraints
- Outputs — defining, sensitive outputs, output dependencies
- Locals — naming prefix & common-tags patterns
- Value types — string, number, bool, list, set, tuple, object, map, null, any
- Input validation — validation blocks & custom conditions
- Building a reusable EC2 module — refactor, test, publish
- **LAB: Convert Week 1 instance → parameterized reusable module**

Module in Terraform

- Provide High-level abstractions over systems composed of multiple components
- Provide the standard way of sharing Terraform code between projects
- A module is a collection of data sources, resources, and any assets (configuration files, templates) that bundled together into reusable component

Modules Overview

Reusable, composable units of Terraform configuration | Ch. 3.1

Root Module

- Every Terraform project is a module
- The directory where you run terraform
- Can call any number of child modules
- Has its own variables, outputs, state

Child Module

- A subdirectory called by the root
- `source = "./modules/ec2-instance"`
- Can call further nested modules
- Scoped: root can't see child internals

Registry Module

- Public Terraform Registry modules
- `source = "terraform-aws-modules/vpc/aws"`
- Version-pinned for reproducibility
- Pre-built, community-maintained

```
module "ec2_instance" {
  source  = "./modules/ec2-instance"   # local path
  # source = "terraform-aws-modules/ec2-instance/aws"  # registry
  version = "~> 5.0"     # for registry modules

  instance_type = var.instance_type    # pass inputs
  name          = "web-server"
}
```

Access outputs: module.ec2_instance.instance_id

`module.<NAME>.<OUTPUT_NAME>`

Module file structure: main.tf, variables.tf, outputs.tf (minimum)

Three Variable Kinds

Input, output, and local — when to use each | Ch. 3.2

variable (Input)

Accepts values from outside the module.
The module's public API — callers set these.

```
variable "instance_type" {
  type        = string
  default     = "t3.micro"
  description = "EC2 instance type"
}
```

Reference input

`var.instance_type`

output (Output)

Returns values to the caller or root.
Exposes internal resource attributes.

```
output "instance_id" {
  value       = aws_instance.web.id
  description = "EC2 instance ID"
  sensitive   = false
}
```

Reference output

`module.ec2.instance_id`

locals (Local)

Named expressions internal to the module.
Never exposed to callers; reduce repetition.

```
locals {
  name_prefix  = "${var.env}-
${var.app}"
  common_tags  = {
    Environment = var.env
  }
}
```

Reference local

`local.name_prefix`

Rule of thumb: use locals to DRY up repetitive expressions; use variables for anything the caller should configure; use outputs to share computed values.

Input Variables

Defining, typing, and securing module inputs | Ch. 3.3

```
variable "db_password" {
  type        = string
  sensitive   = true    # masked in logs
  description = "RDS master password"
  # no default — caller MUST supply
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
  validation {
    condition = contains(
      ["t3.micro","t3.small","t3.medium"],
      var.instance_type)
    error_message = "Must be t3.micro/small/medium"
  }
}
```

type argument

Enforces value type. Without it, any value is accepted. Always use type for robust modules.

default

Makes the variable optional. Omit default to force callers to supply a value.

sensitive = true

Redacts the value from CLI output and plan logs. Does NOT encrypt in state file.

description

Documents the variable purpose. Shown in terraform-docs and IDE hover tips.

validation block

Custom condition (boolean expression). error_message shown when condition is false. Covered Ch. 3.7.

nullable = false

Prevents callers from passing null. Useful when a null default would silently break things.

Outputs & Locals

Exposing values and reducing repetition | Ch. 3.4–3.5

Outputs (Ch. 3.4)

```
output "public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IP of web server"
}

output "db_password" {
  value     = var.db_password
  sensitive = true  # ← propagates sensitivity
}
```

- Outputs become attributes of the module block in the caller
- Root module outputs printed after terraform apply
- Sensitive outputs are redacted in CLI but visible in state
- Output dependencies: depends_on forces ordering
- terraform output <name> reads outputs after apply

Locals (Ch. 3.5)

```
locals {
  # naming prefix pattern
  name_prefix = "${var.env}-${var.project}"

  # common tags pattern
  common_tags = {
    Environment = var.env
    Project     = var.project
    ManagedBy   = "Terraform"
  }
}
```

- Defined once in locals { }, referenced as local.<name>
- Not visible outside the module — purely internal
- Can reference var.*, resource.*, data.*, other locals
- Evaluate lazily — Terraform resolves the dependency graph
- Common use: computed tag maps reused across resources

Value Types

Terraform's full type system | Ch. 3.6

string

```
"t3.micro"
```

UTF-8 text. Interpolation: "${var.name}-server"

number

```
42  /  3.14
```

Integer or float. No quotes.

bool

```
true  /  false
```

Boolean. Used in conditionals and feature flags.

list

```
["us-east-1a","us-east-
1b"]
```

Ordered sequence. Index with [0]. list(string).

set

```
toset(["a","b","c"])
```

Unordered, unique values. Used with for_each.

tuple

```
["t3.micro", 1, true]
```

Ordered, mixed types. Like list but heterogeneous.

object

```
{name="web", port=80}
```

Named attributes, each with own type. object({...}).

map

```
{us-east-1 = "ami-
0abc"}
```

Key-value pairs, same value type. map(string).

null

```
null
```

Absence of a value. Resets optional argument to provider default.

any

```
type = any
```

Disables type checking. Use sparingly — loses safety.

Validating Inputs

Catch bad values before a plan runs | Ch. 3.7

```
variable "environment" {
  type        = string
  description = "Deployment environment"

  validation {
    # condition must evaluate to true/false
    condition = contains(
      ["dev", "staging", "prod"],
      var.environment
    )
    error_message = "environment must be one of dev/staging/prod."
  }
}

variable "vpc_cidr" {
  type = string
  validation {
    condition     = can(cidrnetmask(var.vpc_cidr))
    error_message = "vpc_cidr must be a valid CIDR block."
  }
}
```

condition expression

Any expression that returns bool. Uses var.* to reference the variable being validated.

error_message

Human-readable string shown when condition is false. Must be a literal string.

can() function

Returns true if expression doesn't error. Safe for checking format (CIDR, URL, regex).

Multiple validations

You can add multiple validation {} blocks to one variable — all must pass.

When it runs

Evaluated during plan. Fails fast before any API calls are made.

What it can't do

Cannot reference other variables or resources — only the variable itself.

Building a Reusable Module

Refactoring the Week 1 instance into a portable module | Ch. 3.8

```
my-project/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
└── modules/
    └── ec2-instance/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

```hcl
# root/main.tf
module "web" {
  source        = "./modules/ec2-instance"
  instance_type = var.instance_type
  name          = local.name_prefix
  tags          = local.common_tags
}

output "web_ip" {
  value = module.web.public_ip
}
```

```hcl
# modules/ec2-instance/main.tf
resource "aws_instance" "this" {
  ami           = data.aws_ami.al2.id
  instance_type = var.instance_type
  tags = merge(var.tags, {
    Name = var.name
  })
}
```

1. Define inputs in module's variables.tf

2. Expose useful attributes in module's outputs.tf

3. Call from root with module { source = "..." }

4. Test: terraform plan → verify no changes to intent

5. Publish to registry: requires GitHub + tag vX.Y.Z

HANDS-ON LAB

Week 2 Lab — Parameterize & Modularize the EC2 Config

Objective: Convert the Week 1 instance into a parameterized, reusable EC2 module

1

Add variables.tf

Typed inputs: instance_type, name, tags map
Defaults + sensitive example

2

Add locals

Name prefix pattern: "${var.env}-${var.app}"
common_tags map

3

Add outputs.tf

instance_id, public_dns
Sensitive output example
Output dependencies

4

Add validation

Restrict allowed instance types or environment name values
Verify error message

5

Create module

mkdir modules/ec2-instance
Move resource + data block
Call from root module

6

Stretch: VPC

Use terraform-aws-modules/vpc from public registry
Create VPC for future weeks

Deliverable: Root config calling a custom local module + optional registry VPC module

Week 2 — Key Takeaways

WEEK 2

What you should be able to do after today

Module Structure

Modules are directories with main.tf, variables.tf, outputs.tf. Root calls child with module { source }.

Three Variable Kinds

var.* for inputs (public API), local.* for internal expressions, output for sharing upward.

Type System

Always type your variables. string, number, bool for primitives; list, set, map, object for collections.

Sensitive Values

sensitive = true redacts CLI output but doesn't encrypt state. Manage credentials externally.

Input Validation

validation { condition / error_message } runs at plan time — catch bad values early.

Reusable Modules

Good module = typed inputs, useful outputs, locals for DRY. Call via module { } block with source.

Next week: Expressions, functions, iteration (count/for_each), dynamic blocks → Ch. 4